Informe técnico: Sistema distribuido de pedidos con microservicios

Autor: Paredes Mamani Alberto Orlando
Asignatura: Microservicios
Repositorio: https://github.com/alberthP-M/trabajo-ms-pedidos

1. Resumen del sistema

El proyecto implementa un sistema distribuido de pedidos basado en microservicios, comunicación REST, mensajería asíncrona con Apache Kafka y persistencia en PostgreSQL. El sistema permite autenticarse, consultar productos y existencias, crear pedidos y procesar el cobro mediante una pasarela de pagos simulada.

Los tres servicios principales del dominio son:

- **ms-catalogo:** administra productos, sucursales, precios y existencias.
- **ms-pedidos:** coordina la creación y el ciclo de vida de los pedidos.
- **ms-pagos:** procesa las solicitudes de cobro y las operaciones de reversión.

También se incorporó **ms-auth** como servicio auxiliar de identidad. Este servicio administra usuarios, autenticación, tokens JWT, refresh tokens y cierre de sesión. La interfaz web está implementada con React y permite ejecutar los flujos principales desde el navegador.

La pasarela de pagos se representa mediante un stub configurable. Puede aprobar operaciones, rechazarlas, producir errores, responder lentamente o simular el caso en que autoriza una operación pero pierde la respuesta.

Todo el sistema puede levantarse mediante **docker compose,** incluyendo PostgreSQL, Kafka, los microservicios, el frontend y la pasarela simulada.

2. Arquitectura

Cada servicio posee sus propias reglas de negocio y su propio esquema lógico de datos dentro de PostgreSQL. Se utilizan las bases **auth_db, catalogo_db, pedidos_db** y **pagos_db.** Aunque las bases se ejecutan dentro de una misma instancia de PostgreSQL en el entorno de desarrollo, los servicios no consultan directamente las tablas de los demás contextos.

El frontend React se comunica con los servicios mediante HTTP y utiliza tokens JWT para autenticarse. Las APIs de negocio utilizan el prefijo **/v1/**.

La comunicación síncrona se utiliza cuando se necesita una respuesta inmediata. Por ejemplo, **ms-pedidos** consulta a **ms-catalogo** para reservar existencias. La comunicación asíncrona se utiliza para el procesamiento del cobro:

1. **ms-pedidos** recibe la solicitud de creación del pedido.
2. El pedido se guarda inicialmente en estado **pendiente.**
3. **ms-pedidos** solicita la reserva de los productos a **ms-catalogo.**
4. Si la reserva es correcta, se registra un evento **CobroSolicitado.**
5. El evento se publica en Kafka.
6. **ms-pagos** consume el evento y llama a la pasarela simulada.
7. El resultado se publica mediante Kafka.
8. **ms-pedidos** recibe el resultado.
9. Si el pago fue aprobado, solicita consolidar la reserva y confirma el pedido.
10. Si el pago fue rechazado o falló técnicamente, libera la reserva y marca el pedido como rechazado.

Los topics principales son:

- **payments.commands.v1**
- **payments.results.v1**

La arquitectura se documenta mediante diagramas C4 de contexto y contenedores, además de diagramas de secuencia de la saga.

2.1 Arquitectura

Los servicios HTTP exponen documentación interactiva mediante Swagger UI. La documentación permite consultar los endpoints, parámetros, esquemas de solicitud, respuestas y mecanismos de autenticación JWT.

La documentación se encuentra disponible en los servicios `ms-auth`, `ms-catalogo` y `ms-pedidos`. Además, las especificaciones OpenAPI se entregan como archivos JSON y YAML dentro de la carpeta `docs/openapi/`.

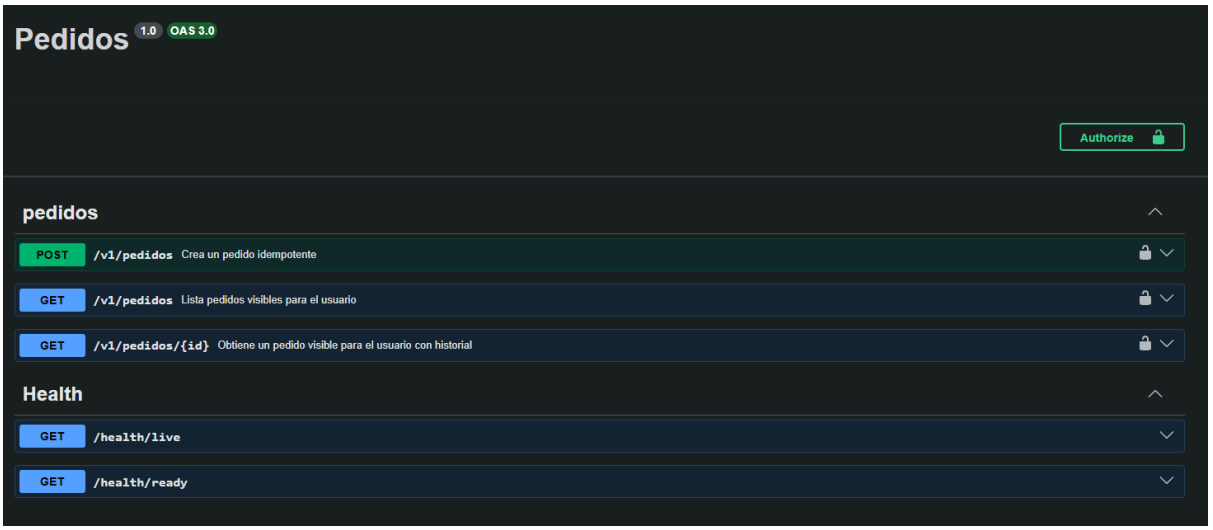


Figura 1. Documentación Swagger de ms-pedidos.

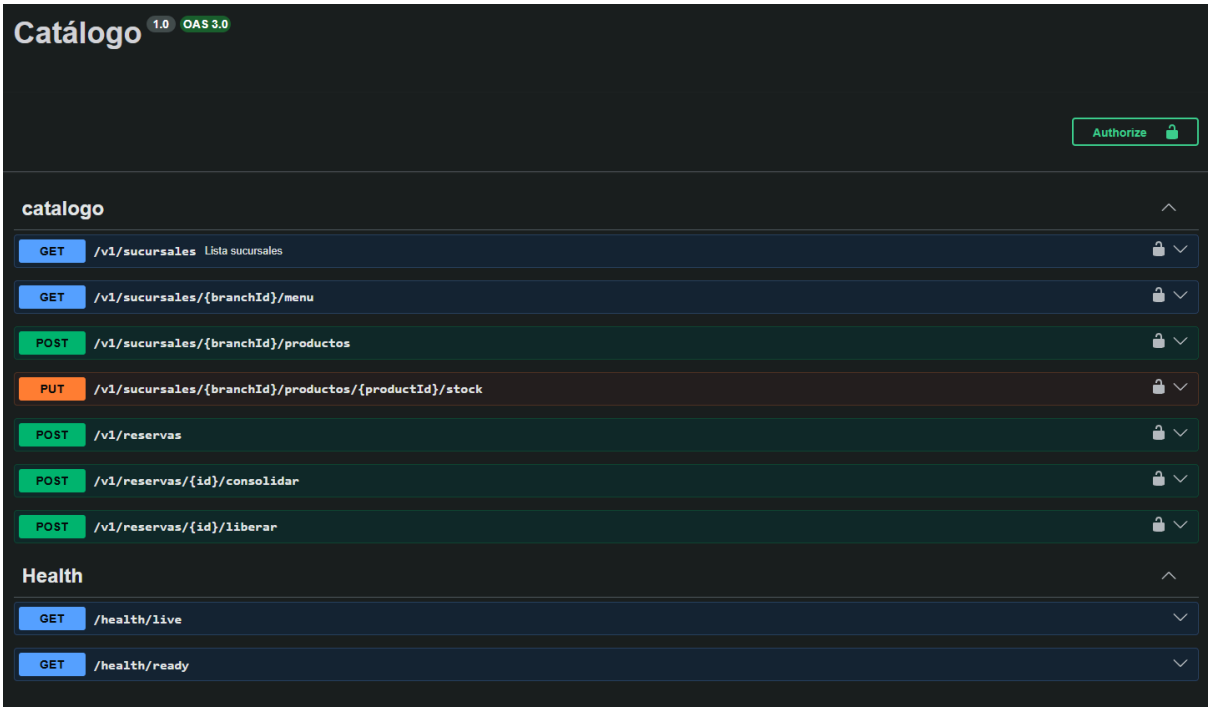


Figura 2. Documentación Swagger de ms-catalogo.

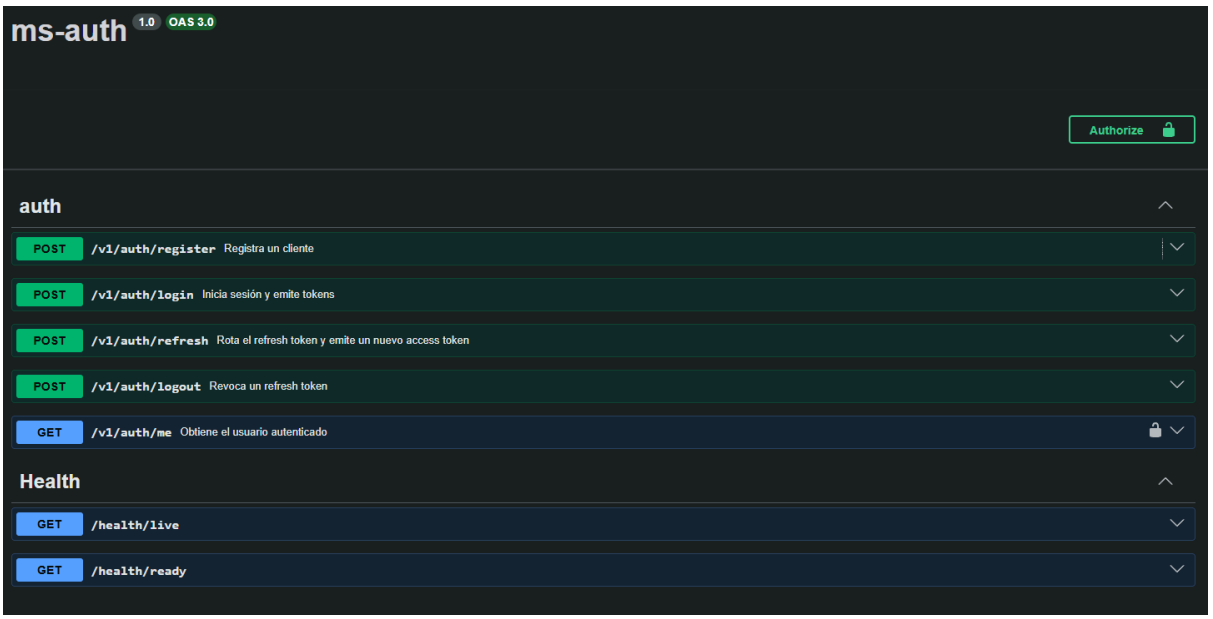


Figura 3. Documentación Swagger de ms-auth.

3. Decisiones técnicas principales

Separación por contextos de negocio

Se separaron Catálogo, Pedidos y Pagos porque cada contexto tiene responsabilidades, datos y reglas de consistencia diferentes. Esta separación evita que un servicio dependa directamente de las tablas internas de otro.

La principal consecuencia es que las operaciones distribuidas no pueden resolverse con una única transacción SQL. Por esta razón, el sistema necesita una saga y mecanismos de compensación.

Saga orquestada

ms-pedidos actúa como coordinador de la saga. El pedido permanece en estado **pendiente** mientras se procesan las operaciones distribuidas. Solo pasa a **confirmado** cuando el cobro fue aprobado y el stock fue consolidado.

Esta decisión facilita explicar el flujo completo desde un único servicio, aunque aumenta la responsabilidad de **ms-pedidos** y obliga a modelar estados intermedios y fallos parciales.

Kafka para el procesamiento de pagos

El cobro se procesa de forma asíncrona porque la pasarela puede ser lenta o no estar disponible. Kafka desacopla la creación HTTP del pedido respecto al resultado final del pago.

El compromiso principal es que el usuario no recibe inmediatamente el resultado final del pedido. Primero recibe un pedido **pendiente** y posteriormente debe consultar su estado.

Kafka no garantiza por sí solo que cada evento se procese exactamente una vez. Por ello, los consumidores utilizan identificadores de evento, claves idempotentes e inbox para evitar repetir efectos.

Outbox, inbox e idempotencia

Los eventos importantes se registran en una tabla outbox dentro de la misma transacción que modifica el estado del negocio. Esto reduce el riesgo de guardar un cambio sin publicar el evento correspondiente.

Los consumidores utilizan inbox y operaciones idempotentes para tolerar la reentrega de mensajes. Además, la creación de pedidos utiliza **Idempotency-Key,** de modo que repetir la misma intención de compra no crea un pedido duplicado.

La pasarela recibe una clave idempotente relacionada con el pedido. Esto evita generar una segunda autorización externa cuando se repite una solicitud debido a un timeout o una reentrega de Kafka.

Resiliencia de la pasarela

ms-pagos utiliza timeout, reintentos con backoff y circuit breaker. Si la pasarela falla repetidamente, el circuito se abre y el pedido termina rechazado con un motivo técnico.

Un timeout no se interpreta automáticamente como ausencia de cobro. La pasarela pudo autorizar la operación antes de que se perdiera la respuesta. Por eso el servicio intenta consultar la operación por su clave idempotente antes de volver a autorizar.

Si el pago fue aprobado pero el catálogo no puede consolidar la reserva, el sistema solicita una reversión. El pedido no se considera completamente finalizado hasta resolver la situación del cobro.

Seguridad y datos de pago

Los usuarios se autentican mediante JWT. Los permisos se aplican según el rol y, en el caso de las sucursales, según el **branchId.**

El sistema no recibe ni almacena números reales de tarjeta. Utiliza tokens opacos de prueba como:

- **tok_demo_aprobado**
- **tok_reject**
- **tok_error**

Estos tokens solo sirven para controlar el comportamiento de la pasarela simulada.

4. Pruebas realizadas

El proyecto contiene pruebas unitarias para autenticación, catálogo, pedidos, pagos y pasarela.

Las pruebas unitarias cubren:

- Inicio de sesión y emisión de tokens.
- Reserva y liberación de existencias.
- Falta de stock.
- Creación idempotente de pedidos.
- Rechazo de pagos.
- Reintentos y circuit breaker.
- Reversión de pagos.
- Comportamiento de la pasarela simulada.

También existe una prueba de integración con PostgreSQL efímero. Esta prueba verifica reserva de stock, idempotencia, liberación, consolidación y creación única de pedidos.

Los escenarios E2E definidos son:

1. Pedido confirmado.
2. Pedido compensado por rechazo de la pasarela.
3. Pedido rechazado por circuito abierto.
4. Pedido rechazado por stock insuficiente.

Durante la verificación se obtuvo compilación correcta, 18 pruebas unitarias aprobadas y una prueba de integración aprobada. Los escenarios E2E deben ejecutarse nuevamente desde un entorno limpio antes de la entrega final, porque los datos persistentes de PostgreSQL afectan el stock entre ejecuciones.



Figura 4. Pruebas e2e

5. Limitaciones conocidas

La implementación tiene las siguientes limitaciones:

- PostgreSQL se ejecuta como una instancia compartida con bases lógicas separadas. No se utilizan servidores físicos independientes por servicio.
- No existe todavía una DLQ funcional para mensajes que fallen repetidamente.
- La readiness verifica principalmente PostgreSQL; no garantiza por sí sola que Kafka o la pasarela estén disponibles.
- El stub de pasarela mantiene sus operaciones en memoria. Al recrearse el contenedor, pierde ese historial.
- Las pruebas E2E dependen del estado persistente del entorno y deben ejecutarse sobre una base limpia para obtener resultados reproducibles.
- No se incorporó un tablero de métricas ni trazas distribuidas completas. La observabilidad se basa principalmente en logs JSON y correlationId.
- Los contratos OpenAPI se generan desde el código mediante Swagger, pero deben exportarse como archivos versionados para que formen parte clara del entregable.
- No se incluyó todavía un pipeline de integración continua.
- **ms-auth** es un servicio adicional respecto a los tres servicios principales solicitados. Se incorporó para centralizar el login y la emisión de JWT, pero no participa directamente en la saga de pedidos.

6. Declaración de uso de inteligencia artificial

Se utilizó un asistente de inteligencia artificial como apoyo durante la planificación, revisión de la arquitectura, identificación de escenarios de fallo, elaboración de documentación y preparación de pruebas.

Las decisiones finales de diseño, la integración de los componentes, la revisión del código y la ejecución de las pruebas fueron responsabilidad del autor. La inteligencia artificial no reemplazó la validación del sistema ni la comprensión de las decisiones técnicas presentadas en este informe.

7. Conclusión

El sistema implementa una arquitectura distribuida con tres servicios principales, comunicación REST y Kafka, persistencia separada, saga orquestada, compensación, idempotencia y mecanismos de resiliencia frente a fallos de la pasarela.

Los flujos principales de confirmación, rechazo, falta de stock y error técnico están implementados y cubiertos por pruebas. El sistema también proporciona logs correlacionados, historial de transiciones y una interfaz web para realizar la demostración.